

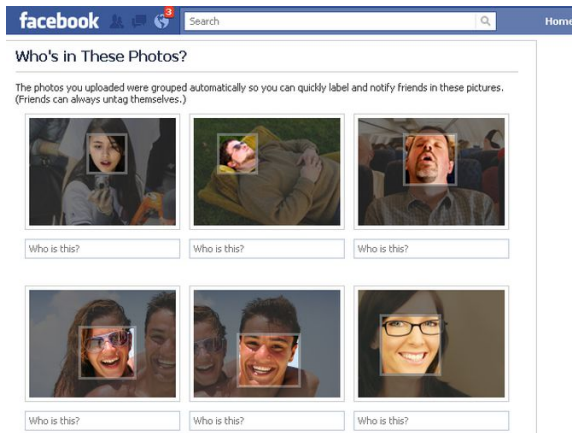
Support Vector Machines

for Face Recognition using `scikit-learn`

J Franco Ray
Scientific Computing Laboratory

November 29, 2016

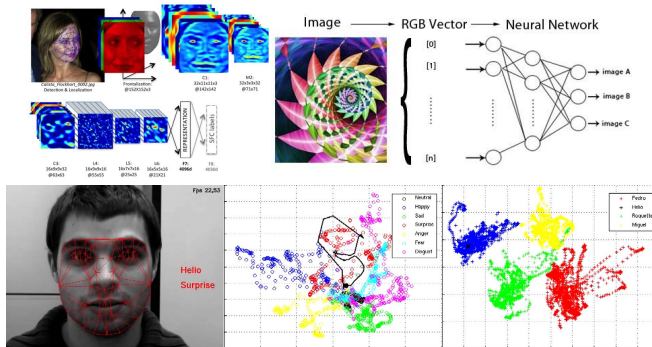
Introduction



Face recognition is one of the applications of using intelligent systems.

<https://www.cnet.com/how-to/how-to-disable-facial-recognition-in-facebook/>

Introduction



http://neuroph.sourceforge.net/image_recognition.html
<https://www.grahamcluley.com/facebook-facial-recognition>

<https://www.youtube.com/watch?v=OpioH4v47N0>

Problem Statement

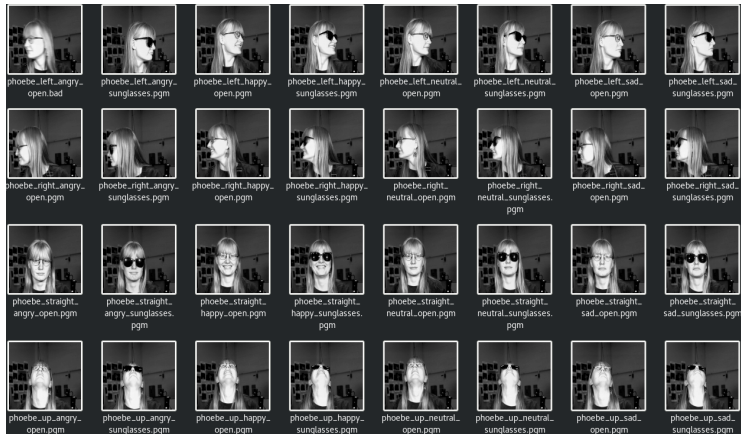
- What are the optimal parameters of the best SVM classifier?
- How is the performance of an optimal classifier?
- To recognize people's face with(out) sunglasses

Dataset



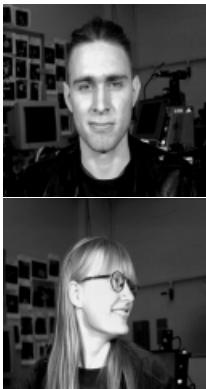
CMU face images data set: 20 persons with 16 poses each

Dataset



CMU face images data set: 20 persons with 16 poses each

Dataset



PGM image specification

P2

24 7

15

0 0 0 0 0 0 0 0 ...

0 3 3 3 3 0 0 7 ...

0 3 0 0 0 0 0 7 ...

0 3 3 3 0 0 0 7 ...

0 3 0 0 0 0 0 7 ...

0 3 0 0 0 0 0 7 ...

0 0 0 0 0 0 0 0 ...

scikit-learn

[Previous](#)
[Home](#) [Installation](#) [Documentation](#) [Examples](#)

This documentation is for
scikit-learn version
0.18.1 — [Other versions](#)

If you use the software,
please consider [citing](#)
[scikit-learn](#).

[sklearn.svm.SVC](#)
Examples using
[sklearn.svm.SVC](#)

Fit me on GitHub

sklearn.svm.SVC

```
class sklearn.svm.SVC(C=1.0, kernel='rbf', degree=3, gamma='auto', coef0=0.0, shrinking=True, probability=False,
to=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=1, decision_function_shape=None,
random_state=None)
```

[\[source\]](#)

C-Support Vector Classification.

The implementation is based on libsvm. The fit time complexity is more than quadratic with the number of samples which makes it hard to scale to dataset with more than a couple of 10000 samples.

The multiclass support is handled according to a one-vs-one scheme.

For details on the precise mathematical formulation of the provided kernel functions and how gamma, coef0 and degree affect each other, see the corresponding section in the narrative documentation: [Kernel functions](#).

Read more in the [User Guide](#).

Parameters: **C** : float, optional (default=1.0)

Penalty parameter C of the error term.

kernel : string, optional (default='rbf')

Specifies the kernel type to be used in the algorithm. It must be one of 'linear', 'poly', 'rbf', 'sigmoid', 'precomputed' or a callable. If none is given, 'rbf' will be used. If a callable is given it is used to pre-compute the kernel matrix from data matrices; that matrix should be an array of shape (n_samples, n_samples).

degree : int, optional (default=3)

Degree of the polynomial kernel function ('poly'). Ignored by all other kernels.

gamma : float, optional (default='auto')

[Previous](#)[Next](#)

scikit-learn

- `gamma` measures how far the influence of a single instance reaches in the multi-dimensional space. *Low values mean 'far'.*
- `C` trades off misclassification against simplicity of the decision surface. *Low C makes it smooth.*

scikit-learn



[Previous](#)
sklearn.model_selection

[Next](#)
sklearn.metrics

[Up](#)
API for sklearn

This documentation is for
scikit-learn version
0.18.1 — [Other versions](#)

If you use the software,
please consider citing
[scikit-learn](#)

[sklearn.model_selection.GridSearchCV](#)
Examples using
[sklearn.model_selection.Grid](#)

[Home](#) [Installation](#) [Documentation](#) [Examples](#)

Fit me on GitHub

sklearn.model_selection.GridSearchCV

```
class sklearn.model_selection. GridSearchCV(estimator, param_grid, scoring=None, fit_params=None, n_jobs=1, iid=True, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score='raise', return_train_score=True)
```

Exhaustive search over specified parameter values for an estimator.

Important members are fit, predict.

GridSearchCV implements a "fit" and a "score" method. It also implements "predict", "predict_proba", "decision_function", "transform" and "inverse_transform" if they are implemented in the estimator used.

The parameters of the estimator used to apply these methods are optimized by cross-validated grid-search over a parameter grid.

Read more in the [User Guide](#).

Parameters: **estimator** : estimator object.

This is assumed to implement the scikit-learn estimator interface. Either estimator needs to provide a `score` function, or `scoring` must be passed.

param_grid : dict or list of dictionaries

Dictionary with parameters names (string) as keys and lists of parameter settings to try as values, or a list of such dictionaries, in which case the grids spanned by each dictionary in the list are explored. This enables searching over any sequence of parameter settings.

scoring : string, callable or None, default=None

A string (see model evaluation documentation) or a scorer callable object / function with signature `scorer(estimator, X, y)`. If None, the `score` method of the estimator is used.

[Previous](#)[Next](#)

scikit-learn

Why do we use open-source libraries?

- More time to analyze the dataset and tuning the parameters instead of implementing the classifier
- Open-source collaboration made the source code more optimized

GCV

```
Grid scores on development set:
0.428 (+/-0.002) for {'kernel': 'rbf', 'C': 1, 'gamma': 'auto'}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.1}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.2}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.30000000000000004}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.4}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.5}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.6}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.7000000000000001}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.8}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 0.9}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1, 'gamma': 1.0}
0.434 (+/-0.009) for {'kernel': 'rbf', 'C': 10, 'gamma': 'auto'}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.1}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.2}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.30000000000000004}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.4}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.5}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.6}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.7000000000000001}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.8}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 0.9}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 10, 'gamma': 1.0}
0.434 (+/-0.009) for {'kernel': 'rbf', 'C': 100, 'gamma': 'auto'}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.1}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.2}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.30000000000000004}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.4}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.5}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.6}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.7000000000000001}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.8}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 0.9}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 100, 'gamma': 1.0}
0.434 (+/-0.009) for {'kernel': 'rbf', 'C': 1000, 'gamma': 'auto'}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.1}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.2}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.30000000000000004}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.4}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.5}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.6}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.7000000000000001}
0.959 (+/-0.000) for {'kernel': 'rbf', 'C': 1000, 'gamma': 0.8}
```

SVM

Kernel	Param	Accuracy (%)
linear		88.7500
polynomial	d=1	88.7500
	d=2	88.7500
	d=3	89.6875
	d=4	90.0000
	d=5	90.6250
	d=6	89.3750
	d=7	85.6250
	d=8	83.1250
	d=9	80.9375
	d=10	77.8125
RBF	$g=1/(128*120)$	29.0625
sigmoid	$g=1/(128*120)$	5.0000

- Accuracy not that sensitive to γ and C
- Support vectors: 291/320, expected result

Conclusion

- SVM with polynomial kernel generally performs better for face recognition problem

Conclusion

- SVM with polynomial kernel generally performs better for face recognition problem
- There are so much methods to apply! See PCA, eigenfaces and deep learning